

6. Python API 기반 프롬프팅 실습

1. OpenAI API 호출 기본 구조

OpenAI의 최신 파이썬 라이브러리에서는 새로운 **OpenAI 클라이언트 객체**를 통해 API를 호출합니다. 먼저 OpenAI 패키지를 임포트하고 **API 키**를 설정해야 합니다. Colab에서는 사전에 저장한 비공개 키를 `google.colab import userdata` 모듈을 통해 불러올 수 있습니다. 그런 다음 `client = OpenAI(api_key=...)` 로 클라이언트를 생성하고, `client.chat.completions.create` 메서드를 사용해 GPT 모델에 프롬프트를 보낼 수 있습니다.

- **모델 선택:** `model="gpt-4o-mini"` 처럼 사용할 모델 이름을 지정합니다 (GPT-4o-mini는 2024년에 공개된 GPT-4o의 경량/저비용 버전입니다). 기본적으로 비용 효율이 높아 다수 호출에 유용하며, 필요시 더 성능이 좋은 `gpt-4o` 로 변경할 수 있습니다.
- **메시지 구성:** `messages=[{"role": "user", "content": "프롬프트"}]` 형태로 대화 메시지를 전달합니다. 시스템 메시지나 어시스턴트 메시지도 추가 가능하지만, 여기서는 기본적으로 사용자 프롬프트만 사용합니다.
- **API 호출 및 응답:** `response = client.chat.completions.create(...)` 로 호출한 후, `response.choices[0].message.content` 에 모델의 답변 텍스트가 들어 있습니다. 또한 `response.usage` 를 통해 이번 호출에서 사용된 **토큰 수** 정보를 얻을 수 있습니다 (프롬프트 토큰, 응답 토큰, 총합).

아래는 OpenAI API 호출의 기본 구조를 보여주는 코드 예시입니다:

##실습 6.1. API 호출을 통한 단일 프롬프트 예시 실습

```
# 한글 폰트 설치
!sudo apt-get install -y fonts-nanum
# 폰트 설치 후 Colab 메뉴에서 런타임/세션 다시 시작을 선택합니다.

!pip install -U openai

from google.colab import userdata
import os

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

import openai
client = openai.OpenAI()

# 단일 프롬프트에 대한 예시 호출
messages = [ {"role": "user", "content": "안녕하세요, 오늘의 날씨는 어떨까요?" } ]
response = client.chat.completions.create(model="gpt-4o-mini", messages=messages)
answer = response.choices[0].message.content # 모델의 응답 내용
print("모델 응답:", answer)
```

해설: 위 코드에서는 `안녕하세요, 오늘의 날씨는 어떨까요?` 라는 간단한 프롬프트를 GPT-4o-mini 모델에 전달하고, 그 응답을 출력합니다. Colab의 `userdata.get` 함수를 통해 OpenAI API 키를 안전하게 불러오고 있으며, `OpenAI()` 클라이언트를 사용함으로써 이후 여러 번 호출을 수행할 때 **API 키와 설정을 반복 입력할 필요 없이** `client` 객체를 재사용할 수 있습니다.

프롬프트에 대한 모델의 응답을 받으면 필요한 경우 이를 변수 (`answer`)에 저장하거나 후속 처리에 활용할 수 있습니다. 다음으로, 이렇게 얻은 응답을 여러 번 반복 호출하거나 성능을 측정하는 방법을 실습해보겠습니다.

2. 여러 개의 프롬프트 반복 실행 자동화

실제 프로젝트에서는 하나의 프롬프트뿐만 아니라 **여러 프롬프트를 자동으로 처리**해야 하는 경우가 많습니다. 이번 단계에서는 미리 정의한 프롬프트 목록을 반복(iterate)하면서, 각 프롬프트에 대해 API를 호출하고 응답을 수집하는 실습을 합니다. 이를 통해 다수의 요청을 일괄 처리하고, 나중에 결과를 비교 분석할 수 있습니다.

예를 들어 몇 가지 질문으로 이루어진 리스트를 만들어 놓고, 이를 순서대로 GPT-4o-mini 모델에 전달해 보겠습니다. 프롬프트 목록은 다양한 유형을 포함하도록 구성하였습니다 (예: 계산 질문, 상식 질문, 설명 요구 등). **for** 루프를 사용해 순회하면서 각 프롬프트의 응답을 받아 출력하고, 리스트에 저장하겠습니다:

##실습 6.2. API 호출을 통한 다중 프롬프트 예시 실습

```
# 여러 개의 프롬프트 리스트 정의
prompts = [
    "1+1은 무엇인가요?",
    "프랑스의 수도는 어디인가요?",
    "햄릿을 쓴 작가는 누구인가요?",
    "파이썬의 리스트와 튜플의 차이점은 무엇인가요?",
    "파이썬의 창시자는 누구인가요?"
]

# 응답을 저장할 리스트 초기화
responses = []

import time # Import the time module

# 각 프롬프트에 대해 API 호출 실행
for prompt in prompts:
    messages = [ {"role": "user", "content": prompt} ]
    response = client.chat.completions.create(model="gpt-4o-mini", messages=messages, temperature=0)
    answer = response.choices[0].message.content
    responses.append(answer)
    print(f"[프롬프트] {prompt}\n[응답] {answer}\n{'-'*40}")
    time.sleep(1) # API 호출 간격 조절 (1초) - Rate Limit 제어
```

코드 설명:

- `prompts` 리스트에 5개의 질문을 문자열로 담았습니다. 다양한 분야와 형식의 질문을 포함해 모델의 응답 특징을 폭넓게 살펴 보겠습니다.
- `for` 루프를 통해 각 `prompt`에 대해 API를 호출합니다. **온도(temperature)**를 0으로 설정하여 응답의 일관성을 높였습니다 (랜덤성을 배제하여 재실행 시에도 같은 답을 얻기 위함).
- 각 응답 `answer`를 `responses` 리스트에 추가하여 저장하고, 동시에 프롬프트와 답변을 출력(`print`)해 확인합니다.

실행 결과 예시: (모델의 실제 응답에 따라 달라질 수 있습니다)

```
[프롬프트] 1+1은 무엇인가요?
[응답] 2

-----

[프롬프트] 프랑스의 수도는 어디인가요?
[응답] 프랑스의 수도는 파리입니다.

-----

[프롬프트] 햄릿을 쓴 작가는 누구인가요?
[응답] 햄릿을 쓴 작가는 영국의 극작가 윌리엄 셰익스피어입니다.

-----

[프롬프트] 파이썬의 리스트와 튜플의 차이점은 무엇인가요?
[응답] 리스트(list)는 대괄호([])를 사용하고 튜플(tuple)은 소괄호(())를 사용합니다. 리스트는 생성 후에 그 값을 변경할 수 있지만 튜플은 한 번 생성하면 값을 변경할 수 없습니다.
```

[프롬프트] 파이썬의 창시자는 누구인가요?

[응답] 파이썬의 창시자는 네덜란드의 프로그래머 귀도 반 로섬(Guido van Rossum)입니다.

위 결과에서 볼 수 있듯이, 다섯 가지 프롬프트에 대해 GPT-4o-mini 모델이 각각 적절한 답변을 생성했습니다. 응답들은 **리스트에 순서대로 저장**되었으므로, 이후 단계에서 이 **responses** 리스트를 사용해 응답의 특성을 분석할 수 있습니다. 이제 각 호출의 **응답 시간**을 측정하여 호출 성능을 살펴보겠습니다.

3. 응답 시간 측정 및 출력

다수의 API 호출을 사용할 때는 **응답 시간(레이턴시)**도 중요한 성능 지표입니다. 이번에는 각 프롬프트를 처리하는 데 걸리는 시간을 측정하여, 어떤 요청이 더 오래 걸리는지 확인해 보겠습니다. 파이썬의 **time** 모듈을 사용하여 시작 시각과 종료 시각을 기록하고 차이를 구하는 방식으로 응답 시간을 계산합니다.

Tip: 응답 시간은 주로 **토큰 수**에 비례하여 증가합니다. 즉, 프롬프트와 출력의 내용이 길어 토큰이 많을수록 API 처리에도 더 시간이 걸립니다.

##실습 6.3. 응답 시간 출력 실습

```
import time

# 응답 시간을 기록할 리스트 초기화
response_times = []

for prompt in prompts:
    start = time.time()          # 시작 시간 기록
    response = client.chat.completions.create(model="gpt-4o-mini",
                                                messages=[{"role":"user","content": prompt}],
                                                temperature=0)
    end = time.time()            # 종료 시간 기록
    elapsed = end - start         # 경과 시간 계산 (초 단위)
    response_times.append(elapsed)
    answer = response.choices[0].message.content
    print(f"[질문] {prompt} → 응답 시간: {elapsed:.2f}초")
    time.sleep(1) # API 호출 간격 조절 (1초)
```

코드 설명: 각 루프에서 **time.time()** 으로 호출 직전 시각을 찍고, 응답을 받은 직후 다시 시간을 잰 다음 둘의 차를 구했습니다. 이렇게 계산된 **elapsed** 시간을 **response_times** 리스트에 저장하고, 포매팅을 통해 소수점 2자리까지 출력했습니다.

예상 출력: (실제 실행 시 네트워크 상황 등에 따라 다를 수 있음)

```
[질문] 1+1은 무엇인가요? -> 응답 시간: 0.35초
[질문] 프랑스의 수도는 어디인가요? -> 응답 시간: 0.80초
[질문] 햄릿을 쓴 작가는 누구인가요? -> 응답 시간: 0.72초
[질문] 파이썬의 리스트와 튜플의 차이점은 무엇인가요? -> 응답 시간: 2.31초
[질문] 파이썬의 창시자는 누구인가요? -> 응답 시간: 0.85초
```

응답 시간 출력으로부터, **프롬프트별 처리 시간의 차이**를 확인할 수 있습니다. 위 예시에서는 네 번째 질문(리스트 vs 튜플)의 응답 생성에 약 2.3초로 가장 오래 걸렸고, 매우 짧은 질문이었던 **1+1**의 답변 생성은 0.3초 정도로 매우 빨랐습니다. 일반적으로 **응답이 길고 내용이 많을수록 (토큰 수가 많을수록) 응답 시간도 증가**하는 경향을 볼 수 있습니다. 다음 단계에서는 실제로 **토큰 사용량**을 추출하여 이런 경향을 수치로 확인해보고, 통계를 계산해보겠습니다.

4. 응답 내 토큰 사용량 추출 및 통계 처리

OpenAI의 API 응답 객체에는 `usage` 필드에 이번 호출에서 사용된 토큰 수가 포함되어 있습니다. `response.usage.prompt_tokens` 와 `response.usage.completion_tokens` 로 프롬프트 입력과 모델 응답에 각각 몇 개의 토큰이 쓰였는지 알 수 있고, `total_tokens` 로 그 합계를 얻을 수 있습니다.

토큰 사용량은 곧 **API 비용** 및 **응답의 글자 수**와 직결되므로, 이를 수집해두면 비용 추산이나 응답 분량 비교에 활용할 수 있습니다. 이번에는 각 프롬프트 호출의 토큰 사용량을 추출하고, 간단한 통계를 계산해보겠습니다.

##실습 6.4. 응답 내 토큰 사용량 추출 및 통계 실습

```
import math

# 토큰 사용량과 응답 저장을 위한 리스트 초기화
results = [] # 각 프롬프트별 결과(dict)를 모아둘 리스트

for prompt in prompts:
    response = client.chat.completions.create(model="gpt-4o-mini",
                                                messages=[{"role":"user","content": prompt}],
                                                temperature=0)
    content = response.choices[0].message.content
    usage = response.usage # 토큰 사용량 정보
    prompt_tokens = usage.prompt_tokens
    answer_tokens = usage.completion_tokens
    total_tokens = usage.total_tokens
    results.append({
        "prompt": prompt,
        "response_text": content,
        "prompt_tokens": prompt_tokens,
        "answer_tokens": answer_tokens,
        "total_tokens": total_tokens
    })
    time.sleep(1) # API 호출 간격 조절 (1초)

# 수집한 토큰 사용량 출력 및 통계 계산
total_token_counts = [res["total_tokens"] for res in results]
print("프롬프트별 토큰 사용량:", total_token_counts)
print("평균 총 토큰 수:", sum(total_token_counts) / len(total_token_counts))
print("최대 총 토큰 수:", max(total_token_counts), "/ 최소 총 토큰 수:", min(total_token_counts))
```

코드 설명:

- 이전과 유사하게 각 프롬프트에 대해 API를 호출하지만, 이번에는 응답의 내용뿐 아니라 `response.usage` 의 세부 항목 (`prompt_tokens`, `completion_tokens`, `total_tokens`)을 추출하여 `results` 리스트에 딕셔너리로 저장했습니다.
- 그런 다음 리스트 내포를 사용해 각 결과의 `total_tokens` 값을 뽑아내어 리스트로 만들고, 이를 기반으로 **평균**, **최대값**, **최소값** 등을 계산했습니다. Python 내장 함수와 연산자를 사용하여 간단히 통계를 낸 모습입니다.

실행 결과 예시:

```
프롬프트별 토큰 사용량: [6, 16, 12, 45, 15]
평균 총 토큰 수: 18.8
최대 총 토큰 수: 45 / 최소 총 토큰 수: 6
```

위 가상의 출력에서, 다섯 개 질문에 대해 사용된 총 토큰 수 리스트는 `[6, 16, 12, 45, 15]` 로 나타났습니다. 가장 작은 6 tokens는 "1+1" 질문에 사용된 것으로 (프롬프트 5 tokens + 응답 1 token 등), 가장 큰 45 tokens는 "리스트와 튜플의 차이점" 질문에 사용된 것입니다. 평균적으로 한 질의응답에 약 18.8 tokens가 쓰였음을 알 수 있습니다. 이처럼 `usage` 데이터를 활용하면 **응답 분량의 크기**를 정확히 수치화할 수 있고, 이를 통해 비용 산정이나 모델의 출력 경향을 분석할 수 있습니다. (실제로 OpenAI API 비용은 사용 토큰 수에 비례하여 청구되므로, 토큰 수 모니터링은 매우 중요합니다.)

추가로, 입력(prompt)과 출력(answer)의 토큰 수를 개별적으로 살펴보는 것도 가능합니다. 다음으로는 이렇게 수집한 데이터를 활용하여 응답 패턴 (길이, 스타일 등)을 비교 분석해보겠습니다.

5. 응답 패턴 비교 분석 실습 (길이, 스타일, 키워드 포함 여부 등)

이제 각 응답의 **패턴**을 여러 측면에서 분석해보겠습니다. 패턴 분석이란, 모델 응답들의 **길이**, **말투/스타일**, **특정 키워드 포함 여부** 등의 특성을 비교하는 것을 말합니다. 이런 분석을 통해 프롬프트에 따른 출력의 **일관성**이나 **차이점**을 파악할 수 있으며, 향후 프롬프트를 개선하거나 모델을 선택하는 데에 근거를 마련할 수 있습니다.

우선, **응답 길이**를 살펴보겠습니다. 앞서 추출한 `prompt_tokens` 와 `answer_tokens` 정보를 통해, 각 질문에 대해 **응답이 프롬프트 대비 얼마나 긴지**를 확인할 수 있습니다. 예를 들어 리스트 vs 튜플 차이를 묻는 질문은 프롬프트 토큰 10개에 출력 토큰 35개로 응답이 매우 길었고, 반면 1+1 같은 단순 질문은 프롬프트 5개에 응답 1개 토큰으로 아주 짧았습니다. 응답 길이를 다른 방식으로 측정한다면, 각 응답 텍스트의 **문자 수**나 **문장 수**를 셀 수도 있습니다.

아래에서는 응답 텍스트의 글자수를 세어보고, 한국어 답변의 경우 존댓말 형식을 사용하고 있는지 (예: "~입니다", "~요"로 끝나는지)도 확인해보겠습니다. 마지막으로 각 응답에 특정 **핵심 키워드**가 포함되어 있는지도 검사해보겠습니다 (예를 들어, 햄릿 작가 질문에 "셰익스피어"가 포함되었는지 등).

##실습 6.5. 응답 패턴 비교 분석 실습

```
for res in results:
    text = res["response_text"]
    length_chars = len(text)          # 응답 글자수
    polite = "예" if ("니다." in text or "요." in text) else "아니오" # 존댓말 여부 판단
    keyword = ""
    if "셰익스피어" in text:
        keyword = "셰익스피어 포함"
    elif "파리" in text:
        keyword = "파리 포함"
    elif "귀도" in text or "Guido" in text:
        keyword = "귀도 포함"
    elif "변경" in text:
        keyword = "변경 관련 내용 포함"
    else:
        keyword = "(특정 키워드 없음)"
    print(f"응답: '{text[:30]}...' | 글자수: {length_chars}, 존댓말 사용: {polite}, 키워드 체크: {keyword}")
```

코드 설명:

- `length_chars` 는 응답 텍스트의 문자열 길이로, 글자 수를 셉니다. (한글도 1글자당 length 1로 계산됩니다.)
- `polite` 변수는 응답에 `"습니다"` 혹은 `"요."` 같은 한국어 존댓말 어미가 포함되어 있는지 단순히 검사하여, 있으면 "예", 없으면 "아니오"로 표시했습니다. 이를 통해 모델이 일관되게 공손한 어조를 쓰는지 확인할 수 있습니다.
- `keyword` 부분은 간단한 조건문으로 각 응답에 예상되는 핵심어가 들어갔는지 확인한 것입니다. 예시로서 셰익스피어, 파리, 귀도(Guido), 변경 등의 단어를 찾고, 발견되면 해당 키워드가 포함되었음을 표시했습니다. (각 키워드는 해당 질문에 대한 정답이나 핵심 개념에 해당합니다.)

예상 출력:

```
응답: '2' | 글자수: 1, 존댓말 사용: 아니오, 키워드 체크: (특정 키워드 없음)
응답: '프랑스의 수도는 파리입니다.' | 글자수: 17, 존댓말 사용: 예, 키워드 체크: 파리 포함
응답: '햄릿을 쓴 작가는 윌리엄 셰익스피어...' | 글자수: 28, 존댓말 사용: 예, 키워드 체크: 셰익스피어 포함
응답: '리스트(list)는 대괄호[], 튜플(tuple)은 소괄호()...' | 글자수: 64, 존댓말 사용: 예, 키워드 체크: 변경 관련 내용 포함
응답: '파이썬의 창시자는 귀도 반 로섬입니다.' | 글자수: 25, 존댓말 사용: 예, 키워드 체크: 귀도 포함
```

분석 결과를 보면, 대부분의 응답이 **공손한 어투(존댓말)**로 작성되었음을 알 수 있습니다 (짧은 2 같은 답변을 제외하면 모두 "~입니다"로 끝맺음). 이는 모델이 기본적으로 한국어 질문에 대한 답변을 정중하게 생성하는 경향이 있음을 보여줍니다. 응답 길이

면에서는 질문마다 큰 차이가 있었는데, 특히 설명형 답변(리스트 vs 튜플)은 60자 이상으로 길고 상세한 반면, 단답형 답변(1+1의 결과)은 1자로 매우 짧았습니다.

키워드 포함 여부를 확인한 결과, 모든 응답이 해당 질문의 핵심 단어를 정확히 언급하고 있었습니다. 예를 들어 "프랑스 수도" 질문에는 답변에 "파리"가 들어있고, "험릿 작가" 질문 답변에는 "셰익스피어"가 포함되어 있습니다. 이처럼 모델이 **정확한 핵심을 언급**하고 있는지 간단한 문자열 검색으로 검증해볼 수 있습니다. (보다 엄밀한 방법으로는 출력 텍스트를 분석하여 정답과의 **정확도**를 계산하는 것이겠지만, 이에 대해서는 다음 단계에서 다룹니다.)

6. Pandas 및 시각화 도구를 통한 결과 표 및 그래프 정리

앞서 수집한 `results` 데이터를 사용해 표 형태로 정리하고, 그래프로 시각화해보겠습니다. **pandas** 라이브러리는 데이터를 표 형태(DataFrame)로 다루기에 편리하며, **matplotlib** 등을 사용해 그래프로 시각화할 수도 있습니다. 이러한 도구를 활용하면 여러 응답 결과를 한눈에 비교할 수 있어, 분석 내용을 효과적으로 전달할 수 있습니다.

먼저, `results` 리스트를 Pandas DataFrame으로 변환하고, 프롬프트별 주요 수치를 표로 확인해보겠습니다:

##실습 6.6. Pandas 및 시각화 도구를 통한 결과 표 및 그래프 실습

```
import pandas as pd

# results 리스트를 DataFrame으로 변환
df = pd.DataFrame(results)
df

# 분석 및 평가 결과 컬럼 추가
df["response_length"] = df["response_text"].apply(len) # 응답 글자수
df

# 정답여부(auto 평가): 각 질문별로 기대하는 키워드/정답이 응답에 포함되었는지 (True/False)
expected_keywords = [
    ["2"], # Q1 정답 "2"
    ["파리"], # Q2 정답 "파리"
    ["셰익스피어"], # Q3 정답 "셰익스피어"
    ["변경"], # Q4 키워드 "변경" (차이점 설명에 포함되어야 할 핵심어)
    ["귀도", "Guido"] # Q5 정답 "귀도" (영문/한글 모두 체크)
]
df["is_correct"] = [ any((kw in res["response_text"]) for kw in expected_keywords[i])
                     for i, res in df.iterrows() ]

df

# 필요한 컬럼만 선택하여 출력
df[["prompt", "prompt_tokens", "answer_tokens", "total_tokens", "response_length", "is_correct"]]
```

코드 설명: `pd.DataFrame(results)` 를 통해 리스트의 딕셔너리들을 표 형태로 변환했습니다. 이렇게 하면 각 키가 열(column)이 되고, 각 딕셔너리가 행(row)이 됩니다.

- `response_length` 열은 응답 텍스트의 글자수를 나타내도록 추가했습니다 (`apply(len)` 사용).
- `is_correct` 열은 간단한 자동 평가 결과로, 미리 정의한 `expected_keywords` 리스트에 있는 정답 키워드들이 응답에 포함되어 있는지 여부를 True/False로 표시했습니다. (리스트 vs 튜플 질문의 경우 "변경"이 포함되었는지를 기준으로 삼는 등, 매우 단순한 체크로 **정답 여부**를 판정했습니다.)
- 마지막에 `print` 로 선택한 몇 개 열만 출력하도록 했습니다 (프롬프트 내용, 토큰 수, 글자수, 정답여부).

DataFrame 출력 예시:

index	prompt	prompt_tokens	answer_tokens	total_tokens	response_length	is_correct
0	1+1은 무엇인가요?	5	1	6	1	True
1	프랑스의 수도는 어디인가요?	8	8	16	17	True
2	햄릿을 쓴 작가는 누구인가요?	8	4	12	28	True
3	파이썬의 리스트와 튜플의 차이점은 무엇인가요?	10	35	45	64	True
4	파이썬의 창시자는 누구인가요?	7	8	15	25	True

위 표에서는 각 질문별로 **프롬프트 토큰**, **응답 토큰**, **총 토큰 수**, **응답 글자수**, **정답 포함 여부**를 정리하여 보여줍니다. 이를 통해 이전에 살펴본 경향을 더욱 명확하게 확인할 수 있습니다:

- 리스트 vs 튜플 차이점 질문(행 3번)의 출력이 토큰 수 26으로 가장 길고 글자수도 64로 가장 큼니다. 반면 1+1 질문(행 0번)은 토큰 6개, 글자수 1로 가장 작습니다.
- `is_correct` 열은 모든 경우에 True로 표시되었는데, 이는 단순 키워드 기반으로 평가한 결과 **모든 답변에 기대한 핵심어가 포함**되어 있음을 의미합니다. (모델이 모든 질문에 대해 비교적 정확한 답을 산출했음을 알 수 있습니다.)

이제 이러한 데이터를 **그래프** 형태로도 시각화해보겠습니다. 아래 그래프는 각 질문에 대해 **프롬프트와 응답 토큰 수**를 막대그래프로 비교한 것입니다:

그래프에서 가장 오른쪽의 **리스트vs튜플** 질문에 주황색 막대(응답 토큰)가 특히 높게 나타나 있는데, 이는 이 질문에 대한 답변이 매우 길었다는 것을 시각적으로 보여줍니다. 반면 **1+1** 질문의 경우 노란색/주황색 막대 모두 거의 보이지 않을 정도로 낮는데, 질문과 답 모두 극히 짧음을 알 수 있습니다. 이러한 시각화는 숫자 표보다도 각 항목 간 차이를 한눈에 파악하기 쉽고, 프롬프트에 따라 **응답 길이 패턴이 어떻게 달라지는지** 직관적으로 이해하는 데 도움이 됩니다.

추가적으로, 응답 시간에 대한 그래프나 정답률에 대한 차트 등을 그릴 수도 있습니다. 예를 들어 응답 시간 vs 총 토큰 수의 관계를 산점도로 나타내면 토큰 수가 많을수록 응답 시간이 길어지는 경향을 확인할 수 있을 것입니다.

입력 토큰과 출력 토큰을 비교하는 막대그래프

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm

# 한글 폰트 설정
font_path = '/usr/share/fonts/truetype/nanum/NanumBarunGothic.ttf' # 나눔바른고딕 경로
fontprop = fm.FontProperties(fname=font_path, size=12)

# 프롬프트별 입력 토큰 vs 출력 토큰 비교 그래프
plt.figure(figsize=(10, 6))

# 막대 폭 및 위치 설정
bar_width = 0.4 # 막대 폭
x_pos_prompt = df.index # 프롬프트 막대 위치
x_pos_answer = df.index + bar_width # 응답 막대 위치 (프롬프트 막대 옆)

plt.bar(x_pos_prompt, df['prompt_tokens'], color='gold', width=bar_width, label='프롬프트 토큰')
plt.bar(x_pos_answer, df['answer_tokens'], color='darkorange', width=bar_width, label='응답 토큰')

plt.xticks(df.index + bar_width/2, df['prompt'], rotation=45, ha='right', fontproperties=fontprop) # X축 눈금 설정 (막대 중앙에 위치)
plt.xlabel('프롬프트', fontproperties=fontprop)
plt.ylabel('토큰 수', fontproperties=fontprop)
plt.title('프롬프트 vs. 응답 토큰 수 비교', fontproperties=fontprop)
```

```
plt.legend(prop=fontprop)
plt.tight_layout()
plt.show()
```

7. 응답 품질 평가 기준 정의 및 자동 평가 예시

프롬프트 엔지니어링에서는 **응답의 품질**을 체계적으로 평가하는 것도 중요합니다. 품질 평가 기준은 과제에 따라 다르겠지만, 일반적으로 **정확성, 완전성, 관련성, 표현력** 등의 항목을 고려할 수 있습니다. 여기서는 간단히 **정확성** 위주로 자동 평가하는 방법을 실습해보았습니다. 위 `is_correct` 열에서 사용한 방식이 바로 한 예인데, 각 질문에 대해 **기대하는 정답 키워드가 출력에 포함되었는지**를 검사하여 정답 여부를 판단했습니다. 이처럼 간단한 방법이지만, 사실상 하나의 **LLM 평가 메트릭(metric)**으로 볼 수 있습니다. 예를 들어 **정확성(Correctness)** 기준은 “모델 출력이 주어진 정답/기대값과 일치하거나 사실적으로 맞는가”를 판단하는 것입니다.

자동 평가의 예를 조금 확장해보면, 다음과 같은 접근이 가능합니다:

- **키워드 매칭:** 우리가 한 것처럼, 정답이나 반드시 포함돼야 할 키워드를 리스트로 정해두고 응답에 해당 단어들이 존재하는지 검사합니다. (예: 수학 문제에서 숫자 '4'를 포함했는지 등)
- **정답 비교:** 답안을 문자열로 비교하거나, 숫자 값의 경우 오차범위 내의 값인지 확인하는 등 정답을 직접 대조합니다. (예: 퀴즈 정답 채점)
- **LLM을 이용한 평가:** 보다 고차원적인 평가는 ChatGPT 같은 **모델을 채점자로 활용**할 수도 있습니다. 즉, 모델에게 “이 응답이 질문에 적절한가?”를 평가하게 하여 점수를 매기게 하는 방법도 연구되고 있습니다. 루브릭을 제공해야 합니다.
- **다양한 품질 지표:** 정확성 외에도 요약의 충실도나 생성된 코드의 실행 가능 여부 등, 과제에 맞는 품질 기준을 세우고 자동화합니다. 예를 들어, 요약 문제라면 Rouge나 BLEU 같은 **텍스트 유사도** 점수를 활용할 수도 있습니다.

우리 실습에서는 키워드 기반으로 평가하여 모든 응답이 True(정답)으로 표시되었지만, 만약 일부 응답이 부정확했다면 False로 표시되었을 것입니다. 그런 경우 개발자는 어떤 프롬프트가 성능이 낮은지 파악하여 **프롬프트를 개선**하거나 **모델을 변경**하는 등의 조치를 취할 수 있습니다. 예컨대, 모델이 특정 질문에 오답을 반복한다면 프롬프트에 힌트를 추가하거나, 필요시 더 강력한 GPT-4o 모델로 호출해 볼 수도 있습니다.

마지막으로, 이렇게 얻은 분석 결과를 파일로 저장해보겠습니다.

8. 실습 결과 데이터 CSV/JSON 저장 방법

분석한 데이터를 **CSV**나 **JSON** 등의 파일로 저장해 두면, 나중에 다시 불러오거나 다른 툴에서 활용하기가 수월합니다. Colab에서 파일로 저장한 후 `** files.download() **`를 통해 로컬로 내려받을 수도 있습니다. 아래는 Pandas를 사용하여 DataFrame을 CSV와 JSON 형식으로 저장하는 방법입니다:

##실습 6.7. 실습 결과 데이터 CSV/JSON 저장 실습

```
# DataFrame을 CSV 파일로 저장 (인덱스 제외)
df.to_csv("results.csv", index=False)

# DataFrame을 JSON 파일로 저장 (UTF-8 인코딩 유지)
df.to_json("results.json", orient="records", force_ascii=False)
```

설명: `to_csv` 메서드는 DataFrame을 콤마로 구분된 값 형식의 문자열로 변환하여 파일로 저장합니다. `index=False`를 주면 행 번호는 저장하지 않습니다. `to_json` 메서드는 JSON 배열 형식으로 저장하며, `force_ascii=False` 옵션을 주어 한글이 유니코드 이스케이프되지 않고 그대로 저장되도록 했습니다. 이렇게 생성된 `results.csv`나 `results.json` 파일에는 앞서 표로 본 내용(프롬프트, 토큰 수, 정답 여부 등)이 모두 담겨 있습니다.

이러한 파일을 저장해 두면, 추후에 **결과 리포트 작성**이나 **추세 분석**을 위해 데이터를 다시 사용하기 쉽습니다. 예를 들어, 여러 실험 설정에 따른 결과 파일들을 모아두고 비교한다거나, 다른 세션(예: Day2)에서 이 데이터를 불러와 활용하는 등의 응용이 가능합니다.

9. 실무 적용 방안

이번 실습에서 다룬 내용들은 실제 현업에서 프롬프트 엔지니어링을 개선하고 모델 활용을 최적화하는 데 유용하게 쓰일 수 있습니다. 정리하면, 다음과 같은 실무 적용 방안을 생각해볼 수 있습니다:

- **대규모 프롬프트 테스트 자동화:** 하나의 최적 프롬프트를 만들기 전에, 후보 프롬프트들을 리스트로 정리하고 일괄 실행하여 응답을 비교해볼 수 있습니다. 이를 통해 어떤 프롬프트가 가장 정확하고 일관된 답을 얻는지 실험적으로 확인 가능합니다.
- **토큰 모니터링을 통한 비용/성능 관리:** API 호출마다 토큰 사용량과 응답 시간을 기록해두면, **서비스 비용 추산**이나 **성능 최적화**에 활용할 수 있습니다. 예를 들어 평균 토큰 수를 바탕으로 월별 예상 비용을 계산하거나, 응답이 지나치게 긴 경우 프롬프트를 조정해 불필요한 내용 생성을 줄이는 식으로 최적화할 수 있습니다.
- **응답 품질 자동 평가 및 경고:** 정의한 품질 기준(키워드 포함 여부 등)으로 응답을 자동 검증하여, 일정 임계 이하의 품질일 경우 알려주는 시스템을 만들 수 있습니다. 예를 들어 챗봇 응답에 중요 키워드가 빠지면 로그를 남기거나 재시도를 트리거하는 등의 **자동 품질 관리**가 가능합니다.
- **모델 간 응답 비교 분석:** GPT-4o-mini vs GPT-4o 처럼 **모델을 바꾸어가며 동일 프롬프트 세트를 실험**하고, 위에서 만든 표와 그래프 기법으로 응답 길이, 정확성 등의 차이를 분석할 수 있습니다. 이를 통해 비용 대비 성능을 평가하여 적절한 모델을 선택하는 근거를 얻습니다.
- **리포팅 및 시각화:** Pandas와 Matplotlib/Plotly를 활용해 **주기적인 성능 리포트**를 생성할 수 있습니다. 예컨대, 하루 동안 사용자 질의에 대한 응답들을 수집하여 평균 응답시간이나 정답률을 도출하고, 이를 대시보드로 시각화하면 서비스 품질을 모니터링할 수 있습니다.

이번 실습을 통해 OpenAI API 사용법부터 시작하여, **데이터 수집 -> 분석 -> 평가 -> 저장**에 이르는 일련의 과정을 경험해보았습니다. 이러한 기법들은 프롬프트 엔지니어링 뿐만 아니라 머신러닝 모델을 활용한 거의 모든 작업에 적용될 수 있으므로, 실무에서도 적극 활용해보시기 바랍니다.